

Contents

Contribution Note	1
Team	1
How responsibilities split	1
Joint (both led together, ~30% of total time)	1
Aryan led (~35% of total time)	2
Ridhi led (~35% of total time)	2
Complete tech stack	2
Languages & runtime	2
Frontend framework & UI	3
Backend framework & libraries	3
LLM providers	3
Data sources & external services	3
Storage	4
DevOps & tooling	4
Frontend env footprint	4
Backend env footprint	4
How we worked	4

Contribution Note

Team

Two-person build.

- Aryan Singh (aryanjsingh) — product thinking + backend agent engineering.
- Ridhi — product thinking + frontend engineering + UX + documentation.

Work split roughly 50/50. Both contributed jointly to product framing, decision-making, demo prep, and bug triage. Roles below describe where each person held the pen on day-to-day work.

How responsibilities split

Joint (both led together, ~30% of total time)

- Track 1 framing and scope decisions.
- Choosing Shopify Catalog MCP over a custom dev store + CSV pipeline.
- Freshness audit as the differentiator (which categories, which signals, where the verdict surfaces).
- Clarify-only-when-it-matters policy.
- Generative-UI-as-grounding principle.
- Per-chat seeded variation strategy (system prompt, search shuffle, suggested actions, greeting).
- Demo storyboard + screenshots + bug triage during dogfooding.
- Final submission docs (PRODUCT, TECHNICAL, DECISIONS, README, this file).

Aryan led (~35% of total time)

- Backend server (backend/src/server.ts): chat streaming, message persistence, vote routes, feed routes, internal-secret gating.
- Shopify Catalog MCP client (backend/src/lib/shopify/catalog.ts): JSON-RPC client, normalization of UCP product/variant shapes, no-store policy compliance.
- Track 1 agent definition and system prompt (backend/src/lib/agents/track1-shopping/): persona, decision tree, tool-sequencing rules, response shape, error recovery, per-chat agentSeed.
- 12 tools: searchProducts, refineSearch, showMore, getProduct, compareProducts, compareSellers, displayProducts, clarifyIntent, assessProductFreshness, webSearch, webFetch, buyProduct.
- Freshness library (freshness/): 19-category detector, curated chipset table (Apple A/M, Snapdragon, Dimensity, Tensor, Exynos, Intel, AMD, Nvidia, AMD RX), non-silicon signals (Wi-Fi 7, BT 5.4, USB-C PD, OLED variants, etc.), verdict composer.
- Web search hardening: Bing → DuckDuckGo → Google fallback chain with Playwright + stealth, 5-min TTL cache, residential proxy support.
- Provider router (backend/src/lib/ai/providers.ts): OpenRouter primary chain + direct DeepSeek with thinking-mode handling.
- Postgres + Redis Docker setup; Drizzle schema + queries used by both processes.
- Self-test harness (backend/scripts/selftest/run.mjs): 8 scenarios, transcript scoring, iteration log.
- Backend low-rating filter and ranking logic.

Ridhi led (~35% of total time)

- Frontend chat shell (frontend/app/(chat)/, frontend/components/chat/): message list, multimodal input, sidebar history, model picker, auth pages with guest CTA.
- hooks/use-active-chat.tsx: useChat wrapper, transport to backend, SWR mutate-on-finish that syncs DB state back into in-memory messages, auto-resume bridge.
- Generative UI parts (frontend/components/chat/shopping/): ProductGrid, ProductCard, ProductDetailCard, ComparisonTable, SellerComparison, FreshnessBadge, GenerationVerdictCard, WebSearchResults, OptionChips, BuyCta.
- Defensive parsing layer (assistant-response-json.ts): JSON envelope unwrap, inline-wrapper extraction, markdown pipe-table stripper, narration stripper, clarify-menu salvage.
- Recommendation feed homepage (frontend/app/page.tsx, frontend/components/feed/): static shell + Suspense streaming, daily-rotated categories, feed search bar.
- Thinking panel UX (MessageThinking, MessageReasoning): collapsed-by-default trace, tool icons, streaming spinner, persisted-tool fallback.
- next-auth setup with guest provider, Drizzle session storage.
- Tailwind + shadcn/ui theming, dark mode, responsive layouts.
- Screenshot capture + walkthrough script for the demo video.

Complete tech stack

Languages & runtime

- TypeScript 5.6 (strict mode across both processes)

- Node $\geq$ 20
- React 19 (frontend) on Next.js 16 App Router with Turbopack
- Single-file Node server (tsx watch) on the backend

Frontend framework & UI

- Next.js 16 App Router
- React 19 + React Server Components
- Tailwind CSS
- shadcn/ui components + Radix UI primitives
- framer-motion for animations
- Lucide icons
- Biome for lint/format
- Playwright for end-to-end tests
- next-auth (credentials provider + guest)
- SWR for cache + revalidation
- Vercel AI SDK 6 (ai, @ai-sdk/react)

Backend framework & libraries

- tsx for dev/start (no build step)
- Vercel AI SDK 6 — streamText, createUIMessageStream, convertToModelMessages, tool()
- Zod for tool input schemas
- Drizzle ORM 0.34 (Postgres)
- postgres client
- redis client (node-redis 5)
- bcrypt-ts for password hashing
- nanoid for IDs
- date-fns for time math

LLM providers

- OpenRouter via @openrouter/ai-sdk-provider
 - Primary: z-ai/glm-4.5-air:free
 - Fallbacks: openai/gpt-oss-120b:free, meta-llama/llama-3.3-70b-instruct:free
 - Title model: meta-llama/llama-3.2-3b-instruct:free
- DeepSeek direct via OpenAI-compatible API (<https://api.deepseek.com>) with deepseek-v4-flash
- @ai-sdk/openai for the OpenAI-compatible adapter
- Optional Vercel AI Gateway path (kept compatible)

Data sources & external services

- Shopify Global Catalog MCP — <https://catalog.shopify.com/api/ucp/mcp> (search_catalog, lookup_catalog, get_product) with UCP agent profile
- Playwright + playwright-extra + puppeteer-extra-plugin-stealth for web search/fetch

- Bing, DuckDuckGo, Google as search providers (fallback chain)
- Optional residential proxy via `SEARCH_PROXY_URL`

Storage

- Postgres 16 (Docker) for users, chats, messages, votes
- Redis 7 (Docker) for feed cache (1h recommendations, 5m search) and web evidence cache
- Drizzle Kit for migrations
- Vercel Blob (optional, for attachment uploads in dev)

DevOps & tooling

- Docker Compose (`postgres:16-alpine`, `redis:7-alpine`) with health-checks
- pnpm (frontend), npm (backend)
- Biome lint + format
- TypeScript `--noEmit` typecheck as the build gate
- Vitest for backend unit tests
- pandoc + xelatex for PDF generation from markdown
- Vercel-ready (`vercel.json`, `vercel-template.json`) but runs entirely local

Frontend env footprint

- `AUTH_SECRET`, `POSTGRES_URL`, `REDIS_URL`
- `OPENROUTER_API_KEY`, `OPENROUTER_REFERRER`
- `SHOPIFY_CATALOG_MCP_URL`, `SHOPIFY_UCP_AGENT_PROFILE`
- `BACKEND_URL`, `BACKEND_INTERNAL_SECRET`
- Optional: `AI_GATEWAY_API_KEY`, `BLOB_READ_WRITE_TOKEN`

Backend env footprint

- `PORT`, `BACKEND_INTERNAL_SECRET`
- `POSTGRES_URL`, `REDIS_URL`
- `OPENROUTER_API_KEY`, `OPENROUTER_REFERRER`
- `DEEPSEEK_API_KEY`, `DEEPSEEK_BASE_URL`
- `SHOPIFY_CATALOG_MCP_URL`, `SHOPIFY_UCP_AGENT_PROFILE`
- Optional: `SEARCH_PROXY_URL`, `SHOPIFY_SHOP`, `SHOPIFY_CLIENT_ID`, `SHOPIFY_CLIENT_SECRET`, `SHOPIFY_ADMIN_ACCESS_TOKEN`, `SHOPIFY_STOREFRONT_ACCESS_TOKEN` (legacy helper scripts only)

How we worked

- Pair-design on the big decisions (catalog source, generative-UI contract, freshness audit shape, clarify policy), solo-execute on the implementation slices.
- `BUILD_PROCESS.md` is the running notebook both authors wrote into during the build. The four submission docs (`PRODUCT`, `TECHNICAL`, `DECISIONS`, this file) are derived from it.
- Decision Log (`DECISIONS.md`) captures every load-bearing choice, including dropped ones — own Shopify dev store, dashboard shell, artifact tools, always-clarify.

- All git authorship under arianj singh for the hackathon submission; pairing happened live so commits do not split per-person.